

C++ Systems Track · G1 Object Model & Lifetime

C++ Systems Track

August 26, 2026

Contents

1	Part 0 · 统一系统模型与访问公式	3
1.1	0.1 G1 的根本问题	3
1.2	0.2 合法访问的系统公式	3
2	Part 1 · 核心六元模型：类型、名称、对象、值、存储与生命周期	3
2.1	1.1 六元基础解耦（以 <code>int x = 42;</code> 为例）	3
2.2	1.2 Object Representation 与 Value	4
2.3	1.3 Object Identity	4
2.4	1.4 Complete Object 与 Subobject	4
2.5	1.5 Object Lifetime 启动与终结	4
2.6	1.6 Scope vs Lifetime	4
2.7	1.7 Storage Duration（存储期）	4
3	Part 2 · 访问路径：指针、引用与别名	5
3.1	2.1 Pointer：作为独立 Object	5
3.2	2.2 解引用与 Pointer Type	5
3.3	2.3 Nullability：C++ 与 Zig 对照	5
3.4	2.4 Aliasing（别名）与 Pointer Ownership	5
3.5	2.5 C++ 与 Zig Const 指针模型对照	5
3.6	2.6 Reference：别名构造，非独立 Object	5
3.7	2.7 Temporary Lifetime Extension	5
3.8	2.8 Zig 为什么不采用 C++ 引用机制？	6
4	Part 3 · 连续数据：数组、退化与视图	6
4.1	3.1 Array is an Array Object, Not a Pointer	6
4.2	3.2 Array-to-Pointer Conversion (Array Decay)	6
4.3	3.3 values vs &values 的步长差异	6
4.4	3.4 Pointer Arithmetic 与 One-Past Pointer	6
4.5	3.5 函数形参的数组调整陷阱	6
4.6	3.6 Zig 数组与切片模型	6
4.7	3.7 <code>std::span</code> 与 Zig <code>[]T</code>	6
5	Part 4 · 四阶段内存与生命周期架构	7
5.1	4.1 四阶段解耦模型	7
5.2	4.2 <code>new</code> 表达式 vs <code>operator new</code>	7
5.3	4.3 Placement New 与存储复用	7
5.4	4.4 <code>std::vector</code> 内部模型	7
5.5	4.5 Zig 的显式内存与清理协议	7

6	Part 5 · 现代值类别与移动语义本质	7
6.1	5.1 表达式值类别划分体系	7
6.2	5.2 核心值类别定义	8
6.3	5.3 <code>std::move</code> 的真实机制	8
6.4	5.4 具名右值引用本身是 <code>rvalue</code>	8
6.5	5.5 函数返回值签名与表达式值类别	8
6.6	5.6 <code>std::move(const T)</code> 的重载决议	8
7	Part 6 · 内存安全缺陷与 <code>Owner / View</code> 审查模型	9
7.1	6.1 悬挂 (Dangling) 缺陷的统一本质	9
7.2	6.2 <code>Vector</code> 扩容导致视图失效全过程	9
7.3	6.3 零拷贝的真实代价	9
7.4	6.4 <code>Owner / View</code> 审查模型	9
7.5	6.5 未定义行为 (UB) 与编译优化	9
7.6	6.6 C++ vs Zig vs Rust 内存安全维度对比	10
8	Part 7 · 验证工具、审查法则与核心铁律	10
8.1	7.1 验证工具的多层防御	10
8.2	7.2 系统级代码审查六步法则 (Code Review Protocol)	10
8.3	7.3 高频核心术语速查	10
8.4	7.4 十二条不可动摇的核心铁律	10

Version: 1.0 (Frozen Review Baseline)

Prerequisite: G0 Native Toolchain & Machine Boundary

Scope: Object / Storage / Value / Type / Lifetime / Pointer / Reference / Array / View / Value Category / Safety

Primary Language: Modern C++23 | Comparison Language: Zig

Purpose: 作为进入 G2 RAII & Ownership 前的长期对象模型与生命周期推理手册

1 Part 0 · 统一系统模型与访问公式

1.1 0.1 G1 的根本问题

C++ 如何在一堆裸地址和 bits 之上建立对象、类型、值、生命周期、指针与引用这些高级语义？



1.2 0.2 合法访问的系统公式

一段合法的 C++ Typed Access, 必须同时满足以下所有约束：

Valid Access = Valid Storage × Active Object Lifetime × Correct Type × Valid Bounds ×
Correct Alignment × Allowed Mutability × Valid Ownership/Borrow Relationship

核心推论：p != nullptr 仅证明指针不为 null，无法证明 *p 合法（仍可能处于未初始化、悬挂、越界、未对齐或类型不匹配状态）。

2 Part 1 · 核心六元模型：类型、名称、对象、值、存储与生命周期

2.1 1.1 六元基础解耦（以 int x = 42; 为例）

- **Type (int)**: 定义合法值域、操作集合、Object Representation、对齐要求与 ABI 属性。
- **Name (x)**: 源码级标识符 (Identifier)，用于在代码中引用实体。Name 不是 Object 本身。
- **Object**: 具有 type、identity/storage/lifetime 语义的 C++ 实体。在 C++ abstract machine 中定义；在 observable implementation 中通常对应某种 storage representation，但 compiler 可在满足 as-if rule 的前提下消除具体物理存储。（C++ Object ≠ OOP Class Instance，标量 int 同样是 Object）。
- **Value (42)**: Object 当前承载的抽象值。（Same Value ≠ Same Object）。
- **Storage**: 承载其 representation 的物理字节空间。（Storage ≠ Object）。
- **Lifetime**: Object 在程序执行过程中实际存活的时间区间。

2.2 1.2 Object Representation 与 Value

Value (42) ≠ Object (x) ≠ Representation (2A 00 00 00)。

Same Bytes ≠ Same Type: 内存中相同的 4 字节 00 00 80 3F, 按 float 解释为 1.0, 按 int32 解释为 1065353216。Bits 本身不携带唯一语义, Type 决定其解释方式。

2.3 1.3 Object Identity

两个独立对象即使值相等 (a == b), 其身份依然不同 (&a != &b)。

2.4 1.4 Complete Object 与 Subobject

```
struct Point { int x; int y; };
Point p; // p 是 Complete Object; p.x 与 p.y 是 Member Subobjects
```

2.5 1.5 Object Lifetime 启动与终结

```
Storage becomes available
↓
Initialization / lifetime-start conditions met
↓
Object lifetime begins
↓
Object Alive
↓
Object lifetime ends (Destruction / reuse)
↓
Storage released or reused
```

关键认知: 对 class 类型, construction 是典型 lifetime-start 机制; 对标量类型, 满足对齐和初始化的存储即可开启生命周期。Storage 存在绝不等于 Object 处于活跃生命周期。

2.6 1.6 Scope vs Lifetime

Scope (作用域): Name 在源码文本中可见的区域。

Lifetime (生命周期): Object 在程序执行过程中实际存活的时间区间。

2.7 1.7 Storage Duration (存储期)

- **Automatic:** 通常与 block/function invocation 相关; 相应 storage duration 结束后, 其 storage 不再为该 object 保留, 常由调用栈机制自然复用。
- **Static:** 全局/静态存储期, 覆盖整个程序执行生命周期。
- **Thread:** thread_local 变量, 与线程生命周期绑定。
- **Dynamic:** 通过动态分配器获取的存储空间, 生命周期由显式操作控制。

3 Part 2 · 访问路径：指针、引用与别名

3.1 2.1 Pointer：作为独立 Object

`int* p = &x;` 中存在两个独立的实体层级：

- `p` 是 Pointer Object（拥有自己的类型 `int*`、Storage、Address `&p` 与 Lifetime）。
- `x` 是 Pointee Object（`p` 的值保存的是 `x` 的地址）。

3.2 2.2 解引用与 Pointer Type

解引用：C++ 为 `*p`，Zig 为 `p.*`。

指针类型决定了寻址步长、解引用类型、对齐假设与别名分析（Aliasing）。指针绝非裸整数地址。

3.3 2.3 Nullability：C++ 与 Zig 对照

- `*C++ (T)**`：允许有效指针或 `nullptr`，非空性主要依赖运行时约定。
- `*Zig (T)**`：类型系统层面强制保证非空；允许为空时必须显式声明为可选指针 `?*T`。
- 共性：两者的非空指针均无法从类型系统上静态排除 Dangling 风险。

3.4 2.4 Aliasing（别名）与 Pointer Ownership

Aliasing：多个访问路径（如 `p` 和 `q`）指向同一底层对象。

Raw Pointer (`T*`) 仅是一条访问路径（Access Path），不编码所有权、生命周期或销毁职责。

3.5 2.5 C++ 与 Zig Const 指针模型对照

语义意图	C++ 语法	Zig 语法	访问与重定向权限
指针可改，内容可改	<code>T* p</code>	<code>var p: *T</code>	指针可重指向，可通过 <code>p</code> 修改目标
指针固定，内容可改	<code>T* const p</code>	<code>const p: *T</code>	指针不可重指向，可通过 <code>p</code> 修改目标
指针可改，内容只读	<code>const T* p</code>	<code>var p: *const T</code>	指针可重指向，只读访问目标
两者皆只读/固定	<code>const T* const p</code>	<code>const p: *const T</code>	全只读、全固定

3.6 2.6 Reference：别名构造，非独立 Object

`int& r = x;`：`r` 是 `x` 的别名。

语言语义：引用在标准中不是 Object（不能建立引用的数组、不能取引用自身的地址、`&r` 得到的是 `&x`）。

不可重绑定：`r = y;` 执行的是赋值操作（`x = y`），绝非将 `r` 重新绑定到 `y`。

3.7 2.7 Temporary Lifetime Extension

`const T& r = temporary;` 等直接绑定存在特定 lifetime-extension 规则（将临时对象的生命周期延长至与引用相同）；不要把它盲目推广到函数返回、间接绑定等其它上下文，具体规则需按语法场景判断。

3.8 2.8 Zig 为什么不采用 C++ 引用机制?

Zig 坚持 Explicit Indirection (显式间接寻址), 函数调用处必须显式写出 `modify(&value)`, 避免 C++ 隐藏传址开销的语法抽象。

4 Part 3 · 连续数据: 数组、退化与视图

4.1 3.1 Array is an Array Object, Not a Pointer

`int values[4];` 的声明类型是 `int[4]` (大小为 16 字节), 是一个包含 4 个元素子对象的数组对象。

4.2 3.2 Array-to-Pointer Conversion (Array Decay)

在多数表达式上下文中, 数组名隐式转换为指向首元素的指针 (`&values[0]`, 类型为 `int*`)。但这只是表达式转换, 数组对象自身从未变成指针。

4.3 3.3 values vs &values 的步长差异

`values` 退化为 `int*`: `values + 1` 偏移 4 字节 (`sizeof(int)`)。
`&values` 类型为 `int(*)[4]`: `&values + 1` 偏移 16 字节 (整个数组的跨度)。

4.4 3.4 Pointer Arithmetic 与 One-Past Pointer

指针算术必须限定在同一数组对象内部及其尾后一位 (One-Past-the-End)。

`end = values + 4` 是合法边界指针, 可用于比较与区间构建 `[begin, end)`, 但绝对不可解引用 `*end`。

4.5 3.5 函数形参的数组调整陷阱

`void foo(int a[100])` 会在编译期自动调整为 `void foo(int* a)`, 100 不构成类型约束。若需强约束必须使用数组引用: `void foo(int (&a)[100])`。

4.6 3.6 Zig 数组与切片模型

类型标记	语义定义
<code>[N]T</code>	定长数组值 (不发生隐式退化)
<code>*[N]T</code>	指向定长数组的指针
<code>[*]T</code>	多项指针 (长度未知的连续缓冲区指针)
<code>T</code>	切片 (Slice = 指针 <code>ptr</code> + 运行时长度 <code>len</code>)

4.7 3.7 std::span 与 Zig []T

`std::span<T>` 与 `Zig []T` 均为 Bounded Non-owning View (带边界信息的非拥有视图)。

Owner vs View: `std::vector` 拥有存储与生命周期; `std::span` 仅借用访问。

核心法则: View 携带 Bounds 绝不等于保证生命周期安全。View 生命周期必须 $\leq$ Backing Storage 生命周期。

5 Part 4 · 四阶段内存与生命周期架构

5.1 4.1 四阶段解耦模型

- | | |
|-----------------|---------------------------------|
| 1. Allocation | : 获取原始未初始化存储 (Raw Storage) |
| 2. Construction | : 在存储上初始化对象, 开启 Object Lifetime |
| 3. Destruction | : 执行资源清理与析构, 结束 Object Lifetime |
| 4. Deallocation | : 将原始存储归还给操作系统或分配器 |

5.2 4.2 new 表达式 vs operator new

`new Widget(...)` (New Expression) = `operator new(sizeof(Widget))` (分配内存) + `Widget` 构造函数 (构造对象)。

`delete p = p->Widget()` (析构对象) + `operator delete(p)` (释放内存)。

5.3 4.3 Placement New 与存储复用

```
alignas(T) std::byte storage[sizeof(T)]; // 仅有 Storage, 无 T 生命周期
T* p = ::new (storage) T(...);           // T 生命周期正式开始
p->~T();                                  // T 生命周期结束, Storage 依然完好
```

推论: 同一块物理存储地址, 可先后承载两个完全不同对象的生命周期。

5.4 4.4 std::vector 内部模型

`capacity`: 已分配的底层 Raw Storage 容量。

`size`: 当前已完成构造并处于活跃状态的 Element Objects 数量。

`clear()`: 析构所有存活元素, 将 `size` 置 0, 但保留底层 Storage 与 Capacity。

5.5 4.5 Zig 的显式内存与清理协议

Zig 不提供语言级隐式构造与析构函数, 通过显式协议解耦:

```
const p = try allocator.create(T); // Allocation
p.* = T.init(...);                // Initialization
defer p.deinit();                  // logical/resource cleanup (ordinary function
    convention)
defer allocator.destroy(p);         // return allocation/storage
```

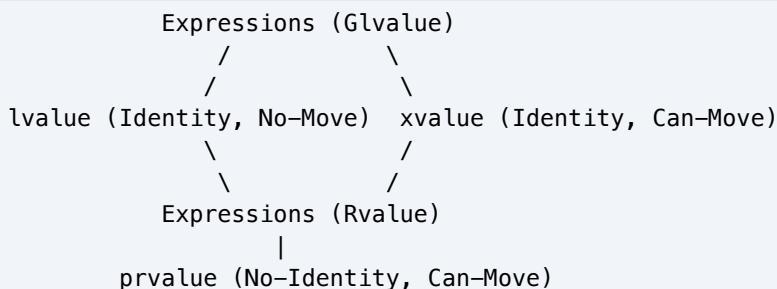
注: Zig 的 `init/deinit` 是普通函数约定, 不等价于 C++ 语言级构造/析构机制。

6 Part 5 · 现代值类别与移动语义本质

6.1 5.1 表达式值类别划分体系

表达式的值类别基于两个正交特征划分:

- **Has Identity**: 表达式是否能够标识某个 object/function identity。
- **Can be Moved From**: 其所绑定的资源是否允许被重用/转移。



(注：上述 Identity / Moveable 划分是帮助系统理解的教学模型，而非 C++ 标准的形式语法定义。)

6.2 5.2 核心值类别定义

- **lvalue**：具有稳定身份标识且不被隐式移动的表达式（如变量名 `x`、返回左值引用的函数调用）。
- **prvalue**：纯粹的值计算结果或初始化式（如字面量 `42`、算式 `a + b`、传值返回的函数调用）。
- **xvalue**：具有明确身份标识、但声明其资源可被重用的即将过期对象（如 `std::move(x)`）。

6.3 5.3 `std::move` 的真实机制

`std::move(x)` 本质是一次无运行时移动动作的 `cast`，结果类型为 `static_cast<std::remove_reference_t<T>&>(x)`，仅将表达式呈现为 `xvalue`，本身不移动任何字节。

真正的资源转移由后续匹配并调用的 Move Constructor / Move Assignment 执行。

Moved-From 状态 ≠ Dangling 状态：Moved-From 对象的生命周期依然处于活跃状态，仍需正常执行析构。

6.4 5.4 具名右值引用本身是 lvalue

```

void foo(T&& r) {
    // 变量 r 的声明类型是 T&&，但表达式 r 是 lvalue（因为 r 拥有标识符 Name）
    bar(std::move(r)); // 若需再次作为右值转发，必须显式使用 std::move
}
  
```

6.5 5.5 函数返回值签名与表达式值类别

函数返回签名	函数调用表达式的值类别	语义定位
<code>T foo()</code>	<code>prvalue</code>	计算结果 / 初始化纯值
<code>T& foo()</code>	<code>lvalue</code>	现有对象的稳定左值引用
<code>T&& foo()</code>	<code>xvalue</code>	现有对象的过期右值引用（可移动）

6.6 5.6 `std::move(const T)` 的重载决议

普通 move constructor `T(T&&)` 无法绑定 `const T&&`；如果存在可行的 `T(const T&)` 等重载，往往会因此选择 copy-like 路径；若无合适重载则会编译失败，类型亦可自行提供 `const T&&` 重载。

7 Part 6 · 内存安全缺陷与 Owner / View 审查模型

7.1 6.1 悬挂 (Dangling) 缺陷的统一本质

访问路径 (Access Path) 依然存在, 但其所引用的底层对象生命周期已终结或存储已被重分配。

```
Use-After-Scope → 指向已退出作用域的局部栈变量
Use-After-Free  → 指向已 delete/free 的堆内存
Double Free     → 多重别名导致同一资源被重复释放
Dangling View   → string_view / span 指向已销毁的临时容器
Iter Invalid    → vector 扩容导致旧迭代器所指内存失效
```

7.2 6.2 Vector 扩容导致视图失效全过程

```
[Before]
Vector Storage: [10][20][30] ← span / pointer / iterator 指向此处

[Action: push_back(40) 触发扩容]
1. 分配新空间 → 2. 拷贝/移动元素 → 3. 析构旧元素 → 4. 释放旧存储空间

[After]
New Storage: [10][20][30][40]
Old View: 仍然指向已释放的原物理地址 → 产生悬挂 (Dangling)!
```

7.3 6.3 零拷贝的真实代价

系统设计准则: *Avoiding a copy usually introduces a lifetime relationship.*

零拷贝绝非单纯减少一次内存复制, 其本质是在组件之间引入了强耦合的生命周期借用协议 (Lifetime Protocol)。

7.4 6.4 Owner / View 审查模型

```
Owner (持有资源所有权: std::vector, std::unique_ptr, std::string)
  | (owns)
  ▼
Object / Backing Storage
  ▲
  | (borrows)
View (借用访问路径: T*, T&, std::span, std::string_view, iterator)
```

核心约束: $\text{Lifetime}(\text{View}) \leq \text{Lifetime}(\text{Owner Backing Storage})$ 。

7.5 6.5 未定义行为 (UB) 与编译优化

UB 不等于立刻崩溃: UB 意味着标准不再对执行行为提供任何保证。它可能表现为数据错乱、静默通过, 或在开启 -O2/-O3 后被编译器优化删除关键分支。

Program didn't crash ≠ Program is valid.

维度	C++23	Zig	Rust
原生指针非空保证	弱 (T* 可为空)	强 (*T 强制非空)	强 (&T 强制非空)
数组自动退化	有 (隐式退化为指针)	无 (严格保持 [N]T 类型)	无 (转换为切片需显式)
连续非拥有视图	std::span<T>	内置切片 []T	切片引用 &[T]
生命周期静态检查	无 (依赖程序员规范)	无 (依赖程序员规范)	强 (编译期借用检查器)
资源清理机制	RAII / 类型驱动	defer / 作用域驱动	RAII / 所有权销毁

7.6 6.6 C++ vs Zig vs Rust 内存安全维度对比

8 Part 7 · 验证工具、审查法则与核心铁律

8.1 7.1 验证工具的多层防御

Language Semantics ≠ Compiler Warnings ≠ Sanitizers ≠ Unit Tests。

```
# 现代 C++ 系统级调试严苛编译配置
clang++ -std=c++23 -O1 -g \
  -Wall -Wextra -Wpedantic \
  -fsanitize=address,undefined \
  -fno-omit-frame-pointer \
  program.cpp -o program
```

AddressSanitizer (ASan): 检测堆/栈 UAF、越界访问、Double Free。

UndefinedBehaviorSanitizer (UBSan): 对多类可动态检查的未定义行为插入运行时检测, 例如部分有符号整数溢出、非法移位、错误对齐、部分无效类型/对象操作等; 它不能覆盖所有 UB, 也不能作为 strict-aliasing 或 lifetime correctness 的完整检测器。

局限: Sanitizer 仅能检测实际执行到的代码路径, 无法形式化证明未覆盖路径的安全性。

8.2 7.2 系统级代码审查六步法则 (Code Review Protocol)

1. 找 **Owner**: 确认谁真正拥有底层资源和存储的生命周期。
2. 找 **Borrowed Access**: 识别所有派生出的 Pointer、Reference、Span、Iterator。
3. 画 **Lifetime Graph**: 验证所有 View 的生存期是否严格包含于 Owner 存活期内。
4. 找 **Invalidators**: 排查 push_back、resize、clear、realloc、erase 等使地址失效的操作。
5. 找 **Ownership Transfer**: 确认移动操作转移的是外层容器所有权还是底层数据指针。
6. 找 **Error Paths**: 在提前 return 或异常分支中, 验证清理机制是否依旧确定性执行。

8.3 7.3 高频核心术语速查

8.4 7.4 十二条不可动摇的核心铁律

1. Object、Storage、Value、Type、Name、Lifetime 是六个相互正交的概念。
2. Storage 的存在绝不证明 Object 仍然处于存活状态。
3. Pointer 是类型化访问路径, 绝不仅仅是一个裸内存地址。
4. Non-null ≠ Valid; 生命周期有效性是访问合法的核心前提。
5. Reference 是别名机制, 在语言语义层绝非独立的指针对象。
6. C++ Array 是完整的数组对象, Array Decay 只是特定表达式下的隐式转换。

英文术语	中文对应	核心系统含义
Object	对象	具有 type、storage 与 lifetime 语义的 C++ 实体
Storage	存储	承载对象二进制表示的物理/虚拟内存区间
Array Decay	数组退化	数组表达式隐式转换为首元素指针的过程
Allocation	分配	仅获取原始存储空间（不开启生命周期）
Construction	构造	在存储上初始化对象并正式开启生命周期
Destruction	析构	执行资源清理并正式终结对象生命周期
Deallocation	释放	将存储空间归还给底层系统或分配器
xvalue	将亡值	具有身份标识、但资源允许被移走的表达式
Dangling	悬挂	访问路径仍存、但目标对象已失效的未定义状态

7. Span / Slice 携带边界信息，但绝不拥有存储，亦不保证生命周期。

8. Allocation ≠ Construction；Destruction ≠ Deallocation。

9. 同一物理内存地址可以先后承载不同对象的生命周期。

10. std::move 不移动任何内存，仅转换表达式为 xvalue；真正移动由移动构造函数执行。

11. 所有 Dangling 缺陷的本质，都是访问路径与底层生命周期契约的不匹配。

12. 系统编程的终极问题：谁拥有？谁借用？谁修改？何时失效？谁负责销毁？

G1 □ G2 的跨越之桥

G1 彻底解决了“对象何时活着与如何访问”，但引发了进阶架构问题：

- Widget* p = new Widget；——究竟由谁负责释放？
- 当遭遇多分支提前 return 或异常抛出时，如何确保无资源泄漏？
- 复杂系统中如何形式化表达单一所有权（Unique Ownership）与共享所有权（Shared Ownership）？

下一站：G2 —RAII & Ownership Architecture

我们将从手动的生命周期管理，跨越到现代 C++ 最具威力与确定性的资源管理架构：析构函数不变量、Rule of Zero/Five、智能指针（unique_ptr / shared_ptr / weak_ptr）以及基于 RAII 的异常安全体系。